

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

END-SEMESTER PROJECT REPORT

on

MAT339 - TRANSFORMER MODELS & SEQUENCE LEARNING

Submitted By

Student Name	:
Register Number	:
Programme	:
Academic Year	: 2026 – 2027
School	: School of Arts, Sciences, Humanities & Education
Project Title	: A Comparative Study of Sequential and Transformer-Based Deep Learning Models for Multi-Class Sentiment Analysis

External Faculty 1 Signature **External Faculty 2 Signature**

Contents

Abstract **6**

1 Introduction **7**
1.1 Background 7
1.2 Natural Language Processing 8
1.3 Deep Learning Models for Sentiment Classification 8
1.4 DistilBERT 8
1.5 Project Overview 9

2 Problem Statement **10**

3 Objectives **11**

4 Dataset Description **12**
4.1 Overview of the Dataset 12
4.2 Data Preprocessing 13
4.3 Sentiment Distribution 13
4.4 Training, Validation and Testing Data 14
4.5 Tokenization and Sequence Representation 14
4.6 Role of the Dataset in the Project 15

5 Methodology **16**
5.1 System Architecture 16
5.2 System Workflow 17
5.3 Data Preprocessing 17
5.4 Tokenization 17
5.5 DistilBERT Tokenizer Initialization 18
5.6 Model Initialization 18
5.6.1 Overview of the Model Architectures 18
5.7 Simple RNN 19
5.8 Long Short-Term Memory 19
5.9 Gated Recurrent Unit 19
5.10 DistilBERT 20
5.11 Classification Architecture 20

5.12 Model Training . . . 21
5.13 Model Evaluation . . . 21

6 Experimental Setup **23**
6.1 Training Configuration . . . 23
6.2 Training Procedure . . . 25
6.3 DistilBERT Initialization and Fine-Tuning . . . 25
6.4 Hardware Configuration . . . 26

7 Evaluation Metrics **27**
7.1 Accuracy . . . 27
7.2 Precision . . . 27
7.3 Recall . . . 28
7.4 F1-Score . . . 28
7.5 Confusion Matrix . . . 28
7.6 Metric Averaging . . . 28
7.7 Computational Evaluation . . . 28

8 Results and Analysis **30**
8.1 Overall Model Performance . . . 30
8.2 Classification Report . . . 31
8.3 Validation Performance . . . 33
8.4 Confusion Matrix Analysis . . . 35
8.5 Training Time, Inference Time, and Model Parameters . . . 36
8.5.1 Training Time Comparison . . . 36
8.5.2 Inference Time Comparison . . . 37
8.5.3 Model Parameter Comparison . . . 37
8.6 DistilBERT Training Progress . . . 38
8.7 Comparative Analysis . . . 39

9 Sample Predictions **40**
9.1 Sample Comment Predictions . . . 40
9.2 Interpretation of the Neutral Example . . . 40

10 Discussion **42**
10.1 Comparison of Model Architectures . . . 42
10.2 Interpretation of Classification Errors . . . 42
10.3 Predictive Performance and Computational Requirements . . . 43
10.4 Interpretation of the Final Model Selection . . . 43

11 Limitations **44**

12 Future Enhancements **46**

13 Conclusion 48

A Source Code 49

References 59

List of Figures

4.1 Sentiment Class Distribution of the Cleaned Dataset 14

5.1 Multi-Class Sentiment Classification Workflow 17
5.2 Architectures of the Simple RNN, LSTM, GRU and DistilBERT Models 18

8.1 Test Accuracy Comparison of the Four Models 31
8.2 Accuracy, Weighted F1-Score and Macro F1-Score of the Four Models 31
8.3 Class-wise F1-Score Comparison 33
8.4 Validation Accuracy Across the Keras Models 34
8.5 Validation Loss Across the Keras Models 34
8.6 Confusion Matrix Comparison of the Four Models 35
8.7 Training Time Comparison of the Four Models 37
8.8 Test-Set Inference Time Comparison of the Four Models 37
8.9 Number of Parameters in Each Model 38
8.10 DistilBERT Fine-Tuning Progress 38

List of Tables

4.1 Original Dataset Information 12
4.2 Dataset Size Before and After Cleaning 13
4.3 Sentiment Distribution in the Cleaned Dataset 13
4.4 Training, Validation and Testing Dataset Distribution 14
4.5 Tokenization and Input Configuration for the Recurrent Models . . . 15

5.1 Simple RNN Architecture 19
5.2 LSTM Architecture 19
5.3 GRU Architecture 20
5.4 Model Training Configuration and Progress 21

6.1 Experimental Training Configuration 24
6.2 Completed Training Epochs and Best Validation Epoch 25
6.3 DistilBERT Training Loss and Validation Performance 25

7.1 Confusion Matrix for Three-Class Sentiment Classification 28

8.1 Overall Model Performance on the Test Dataset 30
8.2 Simple RNN Classification Report 32
8.3 LSTM Classification Report 32
8.4 GRU Classification Report 32
8.5 DistilBERT Classification Report 33
8.6 Validation Accuracy and Loss per Epoch (Keras Models) 34
8.7 Error Decomposition from the Confusion Matrices 35
8.8 Computational Performance of the Four Models 36
8.9 DistilBERT Training Loss and Validation F1-Score 38

9.1 Sample Sentiment Prediction Results 40

Abstract

Sentiment analysis is an important application of Natural Language Processing that enables the automatic identification of opinions expressed in textual data. This case study presents a comparative evaluation of four deep learning models for three-class sentiment classification of user comments: a Simple Recurrent Neural Network (RNN), a Long Short-Term Memory (LSTM) network, a Gated Recurrent Unit (GRU) network, and a fine-tuned DistilBERT model. The study compares their classification performance and computational requirements under a common experimental setting.

The dataset consists of 241,145 raw comments labelled as negative (0), neutral (1) or positive (2). After removing 217 records with missing comments and 26,771 duplicate comments, 214,157 comments are retained and divided using a stratified 80/10/10 split into 171,325 training, 21,416 validation and 21,416 test comments. Text preprocessing, tokenization and sequence padding are applied before model training. The models are evaluated using accuracy, weighted precision, recall and F1-score, macro F1-score, class-wise reports and confusion matrices. Training time, inference time and parameter count are also recorded to assess computational efficiency.

The experimental results show that DistilBERT achieves the highest test accuracy of 87.21% and a weighted F1-score of 87.22%. Among the three recurrent models, LSTM and GRU perform almost identically (84.02% and 83.95% accuracy), while the Simple RNN reaches 81.19%. The GRU requires the least training time per epoch among all models. DistilBERT improves accuracy by about 3.2 percentage points over the best recurrent model, but needs roughly 26 times as many parameters and more than 100 times as much training time per epoch. All models find the negative class the most difficult, and most of their errors involve the neutral class.

The study demonstrates that a pre-trained Transformer provides stronger and more balanced sentiment classification than recurrent models trained from scratch, while compact recurrent architectures remain computationally efficient alternatives. The findings highlight the importance of considering both predictive performance and resource requirements when selecting a deep learning model for sentiment analysis.

Keywords: Sentiment Analysis, Multi-Class Classification, Deep Learning, RNN, LSTM, GRU, Transformer, DistilBERT.

Chapter 1

Introduction

1.1 Background

The rapid growth of digital platforms has led to a substantial increase in user-generated textual content. Social media platforms, discussion forums, product-review sites and messaging services allow individuals to share opinions and experiences with large audiences. These comments provide valuable information about public perception of products, services and technologies, but manually examining large volumes of text is time-consuming and impractical.

Sentiment analysis is a Natural Language Processing (NLP) task that automatically identifies opinions expressed in textual data. In its multi-class form, a comment is assigned to one of several sentiment categories rather than only positive or negative. The three-class formulation used in this study—negative, neutral and positive—is closer to real-world feedback, because many comments are factual or mixed and express no clear polarity. Automated sentiment classification supports customer-feedback analysis, brand monitoring, product improvement and the study of public responses to new technologies.

Traditional machine-learning approaches to sentiment classification rely on representations such as Bag-of-Words, Term Frequency–Inverse Document Frequency (TF-IDF) and n-grams. Although these representations capture useful word-level patterns, they do not adequately model word order, long-range dependencies or the influence of context on sentiment.

Deep learning approaches address these limitations by learning representations directly from token sequences. Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) process text sequentially, while Transformer-based architectures use attention mechanisms to model relationships between all tokens in a sequence. Pre-trained language models such as DistilBERT further support sentiment classification by transferring language knowledge learned during large-scale pre-training to a specific classification task.

1.2 Natural Language Processing

Natural Language Processing is a branch of Artificial Intelligence that enables computers to process, analyze and interpret human language. Its applications include machine translation, text summarization, question answering, information extraction and text classification.

Sentiment analysis is a form of text classification in which a model assigns a sentiment label to an input text. In this case study, the input is a user comment and the output is one of three predefined classes: negative (0), neutral (1) or positive (2). The comments in the supplied dataset have already been lower-cased and stripped of many common words, so the implementation applies only light additional cleaning: lower-casing, removal of HTML line-break tags and web links, and normalization of whitespace. The processed comments are then converted into numerical representations suitable for each model.

1.3 Deep Learning Models for Sentiment Classification

This case study examines four deep learning architectures: a Simple RNN, an LSTM, a GRU and DistilBERT. They differ in how they represent text and how they capture relationships between words.

A Simple RNN processes tokens one at a time and maintains a hidden state that carries information from earlier positions. Because gradients shrink as they are propagated through many time steps, learning long-range dependencies is difficult. LSTM and GRU networks introduce gating mechanisms that regulate the flow of information and ease this problem; the LSTM uses input, forget and output gates together with a separate cell state, whereas the GRU uses only update and reset gates and therefore has fewer parameters.

The RNN, LSTM and GRU models are implemented in TensorFlow/Keras with a trainable embedding layer, learn their word representations from scratch on the training comments, and are trained and evaluated on identical data partitions to support a consistent comparison.

1.4 DistilBERT

DistilBERT is a compressed version of Bidirectional Encoder Representations from Transformers (BERT). It is produced by knowledge distillation and has six Transformer encoder layers instead of twelve, which reduces its size and computation while retaining much of BERT's language-understanding ability. Unlike the recurrent models, which learn representations only from the sentiment dataset, DistilBERT begins

with parameters obtained through language-model pre-training. The pre-trained checkpoint used in this study is

`distilbert-base-uncased`

The checkpoint is loaded with the Hugging Face Transformers library and adapted for three-class classification by specifying three output labels. Its tokenizer converts comments into input token identifiers and attention masks with a maximum sequence length of 128 tokens. DistilBERT is fine-tuned with a custom PyTorch training loop, the AdamW optimizer, a learning rate of 2×10^{-5} with linear warm-up and decay, a batch size of 16 and two epochs, on a GPU-enabled Google Colab runtime.

1.5 Project Overview

The project compares the classification performance and computational requirements of four deep learning models using a dataset of user comments with three sentiment classes. After cleaning, 214,157 unique comments are retained and divided with a stratified split into 171,325 training, 21,416 validation and 21,416 test comments. The complete project follows this pipeline:

1. Load the `sentiment_data.csv` dataset.
2. Examine the dataset structure, missing values and class distribution.
3. Remove missing and duplicate comments and keep the three valid labels.
4. Divide the data into stratified training, validation and test subsets.
5. Preprocess the comment text.
6. Tokenize and pad the comments for the RNN, LSTM and GRU models.
7. Build and train the three TensorFlow/Keras models with early stopping.
8. Tokenize the comments for DistilBERT and fine-tune it using PyTorch.
9. Evaluate all four models using accuracy, precision, recall, F1-score and confusion matrices.
10. Compare parameter counts, training times and inference times.
11. Generate sentiment predictions for sample comments.

In the completed experiment, DistilBERT achieves the highest test accuracy of 87.21% and a weighted F1-score of 87.22%. The comparative analysis considers these results alongside the computational cost of each architecture.

Chapter 2

Problem Statement

The objective of this case study is to develop and evaluate an automated sentiment classification system that identifies the sentiment expressed in user comments. The project compares four deep learning architectures—a Simple Recurrent Neural Network (RNN), a Long Short-Term Memory (LSTM) network, a Gated Recurrent Unit (GRU) network and DistilBERT—to examine their classification performance and computational requirements on the same three-class sentiment task.

Given a comment as input, the system must assign it to one of the following predefined categories:

1. Negative Sentiment
2. Neutral Sentiment
3. Positive Sentiment

The classification problem can be represented mathematically as

$$f_\theta(x) \to y, \qquad y \in \{0, 1, 2\}, \tag{2.1}$$

where x represents the input comment, f_θ represents a trained classification model with parameters θ, and y represents the predicted label: 0 for negative, 1 for neutral and 2 for positive sentiment.

The system must learn sentiment-related patterns from the training comments and apply them to previously unseen comments. Its effectiveness is assessed using accuracy, weighted and macro-averaged precision, recall and F1-score, class-wise classification reports and confusion matrices. Training time, inference time and parameter counts are also compared to examine the computational cost of each architecture. The central question is how recurrent models trained from scratch compare with a fine-tuned pre-trained Transformer in terms of both predictive performance and resource requirements.

Chapter 3

Objectives

The major objectives of the project are:

1. To understand the application of Natural Language Processing and deep learning to multi-class sentiment classification.
2. To study the working principles of Simple RNNs, LSTM networks, GRU networks, Transformer models and DistilBERT.
3. To implement a system that classifies user comments into negative, neutral and positive sentiment categories.
4. To use a dataset of 241,145 user comments for model training, validation and evaluation.
5. To clean the data and convert text into numerical representations suitable for the selected models.
6. To develop and train Simple RNN, LSTM and GRU models using TensorFlow/Keras.
7. To fine-tune the pre-trained `distilbert-base-uncased` model for three-class sentiment classification using PyTorch.
8. To evaluate and compare the four models using accuracy, precision, recall, F1-score, macro F1-score and confusion matrices.
9. To compare the computational requirements of the models using training time, inference time and parameter counts.
10. To analyze the types of classification errors made by each model, particularly for the neutral class.
11. To demonstrate sentiment prediction for new, user-provided comments.

Chapter 4

Dataset Description

4.1 Overview of the Dataset

The dataset used in this case study is supplied as the file `sentiment_data.csv`. It is a collection of English-language user comments, many of which discuss Apple Pay and contactless payments, together with general social-media posts, labelled by sentiment. Each comment is associated with one of three categories: negative, neutral or positive, which makes the dataset suitable for supervised multi-class text classification.

Online comments are short, informal and often ambiguous. They may contain slang, abbreviations, negation, mixed opinions and purely factual statements. Neutral comments in particular have no strong polarity, so they overlap with both positive and negative language. The dataset therefore provides a demanding setting for comparing how well different deep learning models identify sentiment-related patterns.

The raw file contains 241,145 records and three columns: an index column (`Unnamed: 0`), `Comment` and `Sentiment`. The `Comment` column holds the text and the `Sentiment` column holds the label. The index column carries no information and is discarded.

Table 4.1: Original Dataset Information

Attribute	Description
File name	`sentiment_data.csv`
Language	English
Number of records	241,145
Number of columns	3 (index, Comment, Sentiment)
Input feature	`Comment`
Target variable	`Sentiment`
Classification task	Three-class sentiment classification
Missing comments	217

4.2 Data Preprocessing

The cleaning procedure keeps only the `Comment` and `Sentiment` columns, removes records with a missing comment or label, removes duplicate comments and retains only the three valid labels. Duplicates are removed *before* the dataset is split so that identical comments cannot appear in both the training and the test subsets, which would inflate the measured accuracy. The labels are mapped as

$$y = \begin{cases} 0, & \text{Negative sentiment,} \\ 1, & \text{Neutral sentiment,} \\ 2, & \text{Positive sentiment.} \end{cases} \tag{4.1}$$

Of the 241,145 raw records, 217 are removed for missing comments and 26,771 for duplicate comments, leaving 214,157 records.

Table 4.2: Dataset Size Before and After Cleaning

Description	Number of Records
Original dataset	241,145
Removed: missing comments	217
Removed: duplicate comments	26,771
Cleaned dataset	214,157

4.3 Sentiment Distribution

The cleaned dataset contains 48,269 negative, 71,692 neutral and 94,196 positive comments. Unlike a balanced binary dataset, the classes are unequal: positive comments make up about 44% of the data and negative comments only about 22.5%.

Table 4.3: Sentiment Distribution in the Cleaned Dataset

Sentiment	Number of Comments	Percentage
Negative (0)	48,269	22.54%
Neutral (1)	71,692	33.48%
Positive (2)	94,196	43.98%
Total	214,157	100.00%

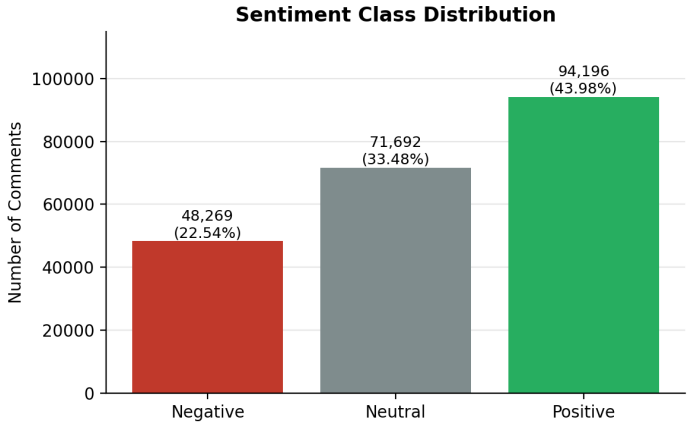


Figure 4.1: Sentiment Class Distribution of the Cleaned Dataset

Because the negative class is the minority, a model can obtain a reasonable overall accuracy while still performing poorly on negative comments. Class-wise precision, recall and F1-score, macro-averaged F1-score and confusion matrices are therefore reported in addition to accuracy.

4.4 Training, Validation and Testing Data

The cleaned dataset is divided into training, validation and test subsets using stratified sampling with a random seed of 42: 80% for training, 10% for validation and 10% for testing. Stratification keeps the class proportions almost identical in all three subsets.

Table 4.4: Training, Validation and Testing Dataset Distribution

Dataset	Negative	Neutral	Positive	Total
Training set	38,615	57,353	75,357	171,325
Validation set	4,827	7,169	9,420	21,416
Testing set	4,827	7,170	9,419	21,416
Total	48,269	71,692	94,196	214,157

4.5 Tokenization and Sequence Representation

Neural networks require numerical input rather than raw text. The implementation uses two different tokenization procedures because the recurrent models and DistilBERT have different input requirements.

For the RNN, LSTM and GRU models, a Keras `Tokenizer` is fitted exclusively on the training comments, so no vocabulary information leaks from the validation or test data. It keeps the 20,000 most frequent words and maps all other words to an

out-of-vocabulary token. Each comment becomes a sequence of integer identifiers, which is then padded or truncated to a fixed length of 128 tokens. Shorter comments are padded with zeros at the end (post-padding) and longer comments are cut to their first 128 tokens (post-truncation).

Table 4.5: Tokenization and Input Configuration for the Recurrent Models

Parameter	Value
Tokenizer	Keras Tokenizer
Vocabulary size	20,000
Out-of-vocabulary token	`<OOV>`
Maximum sequence length	128
Padding method	Post-padding
Truncation method	Post-truncation
Training input shape	(171325, 128)
Validation input shape	(21416, 128)
Testing input shape	(21416, 128)
Embedding dimension	128

A tokenized comment can be written as

$$X = (w_1, w_2, \ldots, w_{128}), \tag{4.2}$$

where each w_i is an integer token identifier or a padding value. The embedding layer of each recurrent model converts these identifiers into dense 128-dimensional vectors. Padding positions are masked (`mask_zero=True`) so that the recurrent layers ignore them.

DistilBERT uses the pre-trained tokenizer of the `distilbert-base-uncased` checkpoint. It splits text into WordPiece sub-word units, which handles rare and misspelled words better than a fixed word vocabulary, and returns input token IDs and an attention mask, padded or truncated to 128 tokens. The 20,000-word vocabulary does not apply to DistilBERT.

4.6 Role of the Dataset in the Project

The dataset provides a common three-class task for comparing sequential processing, gated recurrent mechanisms and transfer learning from a pre-trained Transformer. The shared training, validation and test partitions support a consistent comparison, while the unequal class sizes and the presence of a neutral class allow the study to examine class-wise behaviour and the kinds of errors each model makes. In the completed experiment DistilBERT reaches a test accuracy of 87.21%, while LSTM, GRU and the Simple RNN reach 84.02%, 83.95% and 81.19% respectively.

Chapter 5

Methodology

5.1 System Architecture

The proposed system performs three-class sentiment classification of user comments using four deep learning architectures. Its major components are:

1. **Dataset Loading:** Load `sentiment_data.csv`, containing comments and sentiment labels.
2. **Data Cleaning:** Remove missing and duplicate comments and keep the labels 0, 1 and 2.
3. **Train/Validation/Test Split:** Divide the data into stratified subsets for model development and evaluation.
4. **Text Tokenization:** Convert comments to numerical sequences; the recurrent models use a 20,000-word vocabulary and 128 tokens, and DistilBERT uses its own tokenizer with 128 tokens.
5. **Feature Representation:** Trainable embedding layers in the RNN, LSTM and GRU; contextual representations from the pre-trained encoder in DistilBERT.
6. **Sentiment Classification:** Pass the learned representation through a softmax classification layer that outputs three class probabilities.
7. **Model Training and Fine-Tuning:** Train the three recurrent models with TensorFlow/Keras and fine-tune DistilBERT with PyTorch.
8. **Model Evaluation:** Evaluate all models using accuracy, precision, recall, F1-score, classification reports and confusion matrices.
9. **Computational Comparison:** Compare parameter counts, training times and inference times.

5.2 System Workflow

The workflow begins with loading and cleaning the dataset, followed by the stratified split. The comments are converted into numerical representations and supplied to the respective models. After training, each model predicts the sentiment of the test comments; the predictions are used to compute the classification metrics and confusion matrices, and the models are also compared in terms of training time, inference time and parameter count. Figure 5.1 illustrates the workflow.

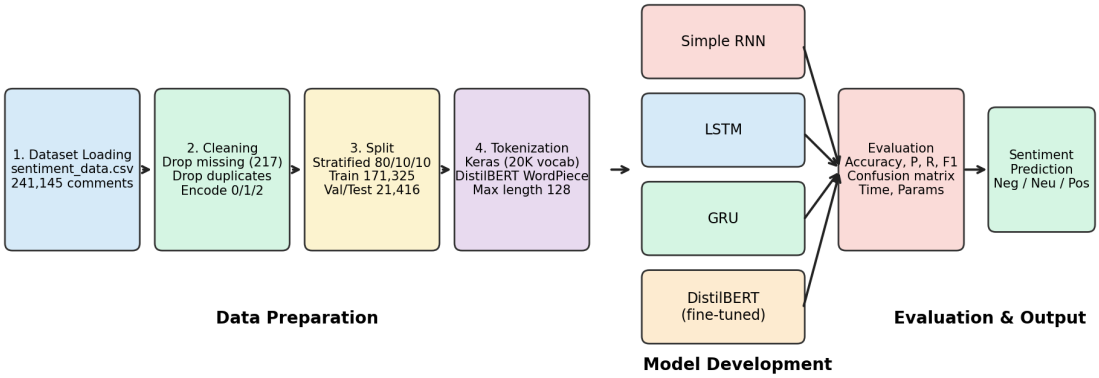


Figure 5.1: Multi-Class Sentiment Classification Workflow

5.3 Data Preprocessing

After cleaning, the comment text is converted to lower case, HTML line-break tags and web links are removed, and repeated whitespace is replaced by a single space. Because the supplied comments were already cleaned (they are lower-case and free of most common function words), the preprocessing is deliberately light, and punctuation is retained for the recurrent models. The preprocessing function is applied separately to the training, validation and test comments.

5.4 Tokenization

The Keras tokenizer for the recurrent models is fitted only on the cleaned training comments. Its configuration and the DistilBERT tokenization procedure are described in Section 4.5. Padded sequences have the shapes (171325, 128), (21416, 128) and (21416, 128) for the training, validation and test subsets.

5.5 DistilBERT Tokenizer Initialization

The DistilBERT tokenizer is initialized from the pre-trained checkpoint with the Hugging Face Transformers library:

```
from transformers import AutoTokenizer

BERT_MODEL_NAME = "distilbert-base-uncased"
bert_tokenizer = AutoTokenizer.from_pretrained(BERT_MODEL_NAME)
```

All training, validation and test comments are tokenized once before training, with truncation and padding to 128 tokens, and stored as tensors. This avoids repeating tokenization inside the data loader at every epoch.

5.6 Model Initialization

Four classification architectures are considered. The recurrent models are constructed as task-specific neural networks with TensorFlow/Keras. DistilBERT is initialized from a pre-trained language-model checkpoint and adapted to three-class classification with a task-specific classification head.

5.6.1 Overview of the Model Architectures

Figure 5.2 illustrates the architectures. The Simple RNN, LSTM and GRU share the same structure—embedding, one recurrent layer with 64 units, dropout and a three-unit softmax layer—and differ only in the type of recurrent layer. DistilBERT uses its own tokenizer, six Transformer encoder layers with a hidden size of 768, a pre-classifier layer and a three-unit classifier.

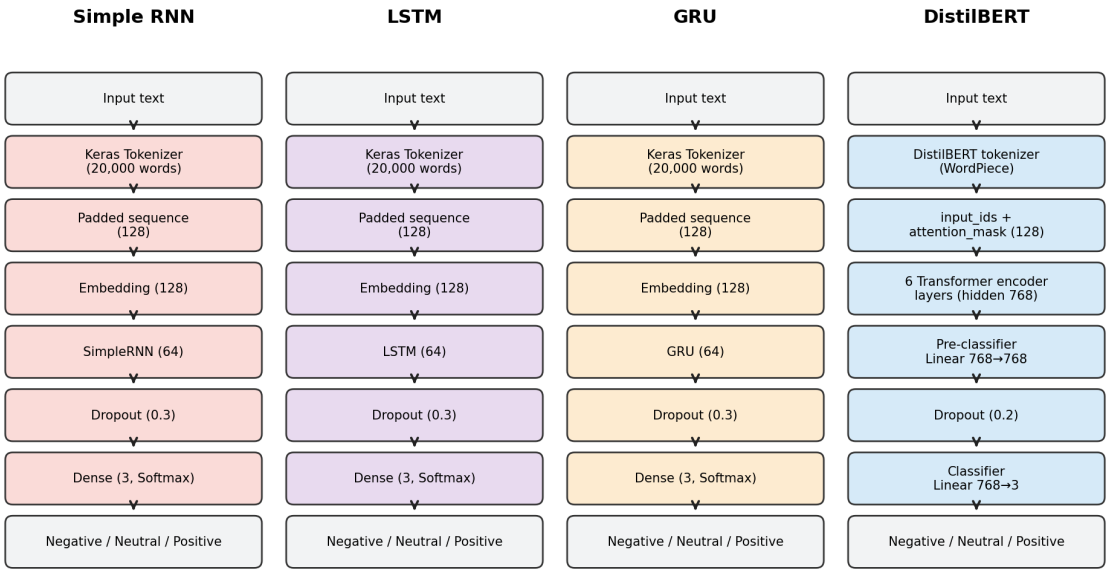


Figure 5.2: Architectures of the Simple RNN, LSTM, GRU and DistilBERT Models

5.7 Simple RNN

The Simple RNN processes the tokens sequentially and keeps a hidden state that carries information from earlier positions in the comment. It consists of an embedding layer, a SimpleRNN layer with 64 units, a dropout layer with rate 0.3 and a dense softmax output layer with three units.

Table 5.1: Simple RNN Architecture

Layer	Output Shape	Parameters
Embedding	(128, 128)	2,560,000
SimpleRNN	64	12,352
Dropout (0.3)	64	0
Dense output (softmax)	3	195
Total		2,572,547

5.8 Long Short-Term Memory

The LSTM replaces the recurrent layer with an LSTM layer of 64 units. Its input, forget and output gates and its separate cell state regulate what information is stored, discarded and exposed, which helps the model retain relevant information across longer comments.

Table 5.2: LSTM Architecture

Layer	Output Shape	Parameters
Embedding	(128, 128)	2,560,000
LSTM	64	49,408
Dropout (0.3)	64	0
Dense output (softmax)	3	195
Total		2,609,603

5.9 Gated Recurrent Unit

The GRU uses a gated recurrent layer with 64 units. Its update and reset gates control the flow of information; with no separate cell state it has about 25% fewer recurrent parameters than the LSTM and is cheaper to compute.

Table 5.3: GRU Architecture

Layer	Output Shape	Parameters
Embedding	(128, 128)	2,560,000
GRU	64	37,248
Dropout (0.3)	64	0
Dense output (softmax)	3	195
Total		2,597,443

5.10 DistilBERT

DistilBERT is initialized from the pre-trained `distilbert-base-uncased` checkpoint and adapted to three-class classification:

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=3)
```

The model contains 66,955,779 trainable parameters: a 30,522-word embedding table of dimension 768, six Transformer blocks (each with multi-head self-attention and a 3,072-unit feed-forward network), a pre-classifier layer ($768 \to 768$) and a classifier layer ($768 \to 3$). When the checkpoint is loaded, the pre-classifier and classifier weights are reported as newly initialized because they are not part of the pre-trained checkpoint; they are learned during fine-tuning together with the adapted encoder weights. A custom PyTorch training loop is used instead of the Hugging Face `Trainer` interface.

5.11 Classification Architecture

In all four models the learned comment representation h is passed to a final layer that produces three logits:

$$z = Wh + b, \qquad z \in \mathbb{R}^3. \tag{5.1}$$

The softmax function converts the logits into class probabilities,

$$P(y = i \mid X) = \frac{e^{z_i}}{\sum_{j=0}^{2} e^{z_j}}, \qquad i \in \{0, 1, 2\}, \tag{5.2}$$

and the predicted sentiment is the class with the highest probability,

$$\hat{y} = \arg\max_{i \in \{0,1,2\}} P(y = i \mid X). \tag{5.3}$$

The recurrent models use sparse categorical cross-entropy, and DistilBERT uses the cross-entropy loss of its sequence-classification head. Both take integer class labels.

5.12 Model Training

The RNN, LSTM and GRU are trained with the Adam optimizer (learning rate 10^{-3}) and sparse categorical cross-entropy loss, with a batch size of 128 and a maximum of five epochs. Validation loss is monitored by two callbacks: early stopping with a patience of two epochs, which restores the weights of the best epoch, and learning-rate reduction by a factor of 0.5 after one epoch without improvement (minimum 10^{-6}). The models stop after different numbers of epochs: the RNN and LSTM complete four epochs and the GRU completes two.

DistilBERT is fine-tuned with the AdamW optimizer (learning rate 2×10^{-5}), a linear learning-rate schedule with 10% warm-up, gradient clipping at a norm of 1.0, a batch size of 16 and two epochs. After each epoch the validation weighted F1-score is computed, and the weights with the highest validation F1-score are restored before testing. The recorded training losses are 0.5068 and 0.3432 and the validation F1-scores are 0.8616 and 0.8722.

Table 5.4: Model Training Configuration and Progress

Model	Batch Size	Configured Epochs	Completed Epochs
Simple RNN	128	5	4
LSTM	128	5	4
GRU	128	5	2
DistilBERT	16	2	2

The measured training times are 86.27 s for the Simple RNN, 69.22 s for the LSTM, 27.90 s for the GRU and 4,394.1 s (about 73 minutes) for DistilBERT.

5.13 Model Evaluation

All four models are evaluated on the same held-out test subset of 21,416 comments (4,827 negative, 7,170 neutral and 9,419 positive). The following measures are used:

1. **Accuracy:** the proportion of correctly classified comments.
2. **Precision, Recall and F1-score:** computed for each class and averaged (weighted and macro).
3. **Classification Report:** precision, recall, F1-score and support for each class.
4. **Confusion Matrix:** a 3×3 summary of correct and incorrect predictions.

5. **Computational Measures:** training time, inference time and number of parameters.

The test accuracies are 81.19% for the Simple RNN, 84.02% for the LSTM, 83.95% for the GRU and 87.21% for DistilBERT. The validation subset is used only for early stopping and checkpoint selection, so the test subset is not used to choose any model setting.

Chapter 6

Experimental Setup

6.1 Training Configuration

The experiment compares four deep learning architectures on three-class sentiment classification. The three recurrent models are implemented with TensorFlow/Keras and DistilBERT with PyTorch and the Hugging Face Transformers library. All models use the same cleaned data and the same stratified split.

Table 6.1: Experimental Training Configuration

Parameter	Value
Dataset	`sentiment_data.csv`
Classification task	Three-class sentiment classification
Number of classes	3 (Negative, Neutral, Positive)
Original samples	241,145
Samples after cleaning	214,157
Training samples	171,325
Validation samples	21,416
Testing samples	21,416
Data split	Stratified 80/10/10
Random seed	42
Recurrent models	Simple RNN, LSTM, GRU
Pre-trained model	DistilBERT (`distilbert-base-uncased`)
Vocabulary size (recurrent models)	20,000
Maximum sequence length	128 tokens
Embedding dimension (recurrent models)	128
Recurrent units	64
Dropout rate (recurrent models)	0.3
Recurrent-model batch size	128
Maximum epochs (recurrent models)	5
Training steps per epoch (recurrent)	1,339
Recurrent-model optimizer	Adam (learning rate 10^{-3})
Recurrent-model loss	Sparse categorical cross-entropy
Early stopping	Validation loss, patience 2, best weights restored
Learning-rate reduction	Factor 0.5, patience 1, minimum 10^{-6}
DistilBERT batch size	16
DistilBERT epochs	2
DistilBERT steps per epoch	10,708
DistilBERT optimizer	AdamW (learning rate 2×10^{-5})
DistilBERT schedule	Linear, 10% warm-up
Gradient clipping	Maximum norm 1.0
Checkpoint selection	Best validation weighted F1-score
TensorFlow version	2.21.0
Training device	GPU (Google Colab runtime)

6.2 Training Procedure

The experiment begins by loading and cleaning the dataset. The cleaned comments are split into training, validation and test subsets, and the text is converted into the numerical form required by each architecture. The three recurrent models are trained with callbacks that stop training when the validation loss no longer improves; the weights of the epoch with the lowest validation loss are restored before testing.

Table 6.2: Completed Training Epochs and Best Validation Epoch

Model	Configured	Completed	Lowest Val. Loss
Simple RNN	5	4	Epoch 2 (0.5216)
LSTM	5	4	Epoch 2 (0.4393)
GRU	5	2	Epoch 2 (0.4461)
DistilBERT	2	2	Highest val. F1 at epoch 2

The Simple RNN and LSTM both reach their lowest validation loss at the second epoch, after which validation loss rises while training accuracy continues to increase (to 92.96% and 90.07% at epoch 4), a clear sign of overfitting that early stopping correctly guards against. The GRU run ended after two epochs in the reported experiment; because its training time covers fewer epochs, per-epoch time is also reported for a fair speed comparison.

6.3 DistilBERT Initialization and Fine-Tuning

DistilBERT is initialized from the pre-trained checkpoint with a three-label classification head. Comments are tokenized once with the associated tokenizer and padded or truncated to 128 tokens. Fine-tuning uses the AdamW optimizer with a learning rate of 2×10^{-5}, a linear warm-up and decay schedule, a batch size of 16 and two epochs.

Table 6.3: DistilBERT Training Loss and Validation Performance

Epoch	Training Loss	Validation Weighted F1
1	0.5068	0.8616
2	0.3432	0.8722

After both epochs, the checkpoint with the best validation F1-score (epoch 2) is evaluated on the 21,416-comment test subset, giving an accuracy of 87.21%, weighted precision of 87.29%, weighted recall of 87.21% and weighted F1-score of 87.22%. The recorded training time is 4,394.1 seconds.

6.4 Hardware Configuration

The implementation checks the available computing devices before training. TensorFlow 2.21.0 detects a GPU, and PyTorch selects the CUDA device for DistilBERT:

```
import torch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch device:", device)     # -> cuda
```

The experiment was run in a GPU-enabled Google Colab environment. The reported training and inference times reflect these execution conditions and may vary with hardware, software versions and system load. The recurrent models and DistilBERT also run in different frameworks (TensorFlow versus PyTorch) with different batch sizes, so their times indicate order-of-magnitude differences rather than exact ratios.

Chapter 7

Evaluation Metrics

The performance of the sentiment classification models is evaluated using accuracy, precision, recall and F1-score, together with confusion matrices. Because this is a three-class problem, the target classes are negative (0), neutral (1) and positive (2). All models are evaluated on the same test set of 21,416 comments. For each class c (treated as the "positive" class in a one-versus-rest view), the following quantities are used:

- True Positives (TP_c): comments of class c correctly predicted as c.
- False Positives (FP_c): comments of other classes predicted as c.
- False Negatives (FN_c): comments of class c predicted as another class.

7.1 Accuracy

Accuracy is the proportion of comments assigned the correct label:

$$\text{Accuracy} = \frac{\sum_c TP_c}{N}, \tag{7.1}$$

where N is the number of test comments. Because the classes are unequal, accuracy alone does not show how errors are distributed across classes.

7.2 Precision

Precision for a class is the proportion of comments predicted as that class that truly belong to it:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}. \tag{7.2}$$

7.3 Recall

Recall for a class is the proportion of the comments of that class that the model identifies correctly:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}. \tag{7.3}$$

7.4 F1-Score

The F1-score is the harmonic mean of precision and recall:

$$F1_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{7.4}$$

7.5 Confusion Matrix

A confusion matrix summarizes the number of correct and incorrect predictions. Rows represent the actual sentiment and columns the predicted sentiment. For three classes it is a 3×3 matrix whose diagonal contains the correct predictions.

Table 7.1: Confusion Matrix for Three-Class Sentiment Classification

	Predicted		
Actual	Negative	Neutral	Positive
Negative	correct	N→Neu	N→Pos
Neutral	Neu→N	correct	Neu→Pos
Positive	Pos→N	Pos→Neu	correct

7.6 Metric Averaging

Two averages of the class-wise scores are reported. The *weighted* average weights each class by its number of test comments, so it reflects overall performance on the actual class mix and is close to accuracy. The *macro* average gives each class equal weight and therefore penalizes weak performance on a small class such as the negative class:

$$F1_{\text{macro}} = \frac{1}{3}\sum_{c=0}^{2} F1_c, \qquad F1_{\text{weighted}} = \sum_{c=0}^{2} \frac{n_c}{N} F1_c. \tag{7.5}$$

7.7 Computational Evaluation

The computational cost of each model is measured by three quantities: the *training time* (wall-clock time of the fitting procedure), the *inference time* (wall-clock time

to predict the whole test set) and the number of trainable *parameters*. Because the models stop after different numbers of epochs, the training time per epoch is also reported.

Chapter 8

Results and Analysis

8.1 Overall Model Performance

Table 8.1 summarizes the performance of the four models on the 21,416-comment test set. Precision, recall and F1-score are weighted averages over the three classes; the macro F1-score is also given because it weights the minority negative class equally.

Table 8.1: Overall Model Performance on the Test Dataset

Model	Accuracy	Precision	Recall	F1-Score	Macro F1
Simple RNN	81.19%	81.10%	81.19%	81.02%	79.71%
LSTM	84.02%	84.11%	84.02%	83.93%	82.92%
GRU	83.95%	83.91%	83.95%	83.85%	82.89%
DistilBERT	**87.21%**	**87.29%**	**87.21%**	**87.22%**	**86.47%**

DistilBERT achieves the highest value on every metric. Its accuracy is 3.19 percentage points higher than the LSTM, 3.26 higher than the GRU and 6.02 higher than the Simple RNN. The LSTM and GRU are almost identical: the LSTM classifies 17,993 test comments correctly and the GRU 17,978, a difference of only 15 comments (0.07 percentage points), which is too small to call one model better from a single run. The Simple RNN is clearly the weakest model.

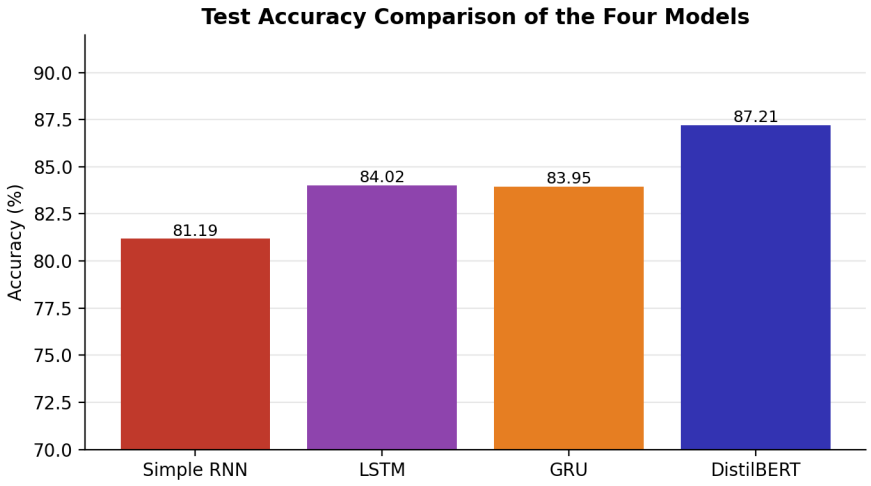


Figure 8.1: Test Accuracy Comparison of the Four Models

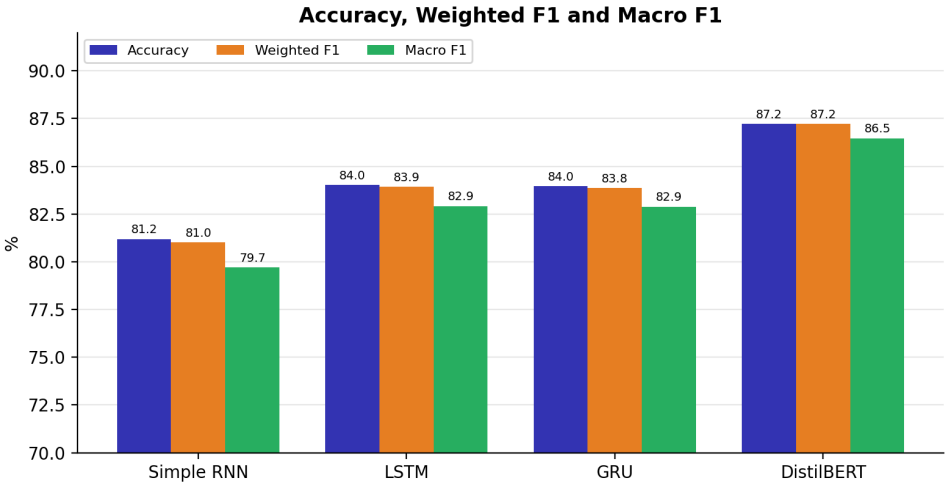


Figure 8.2: Accuracy, Weighted F1-Score and Macro F1-Score of the Four Models

8.2 Classification Report

The class-wise reports show where each model succeeds and fails.

Table 8.2: Simple RNN Classification Report

Category	Precision	Recall	F1-score	Support
Negative	0.7847	0.6735	0.7249	4,827
Neutral	0.7962	0.8448	0.8198	7,170
Positive	0.8358	0.8577	0.8466	9,419
Accuracy			0.8119	21,416
Macro avg.	0.8056	0.7920	0.7971	21,416
Weighted avg.	0.8110	0.8119	0.8102	21,416

Table 8.3: LSTM Classification Report

Category	Precision	Recall	F1-score	Support
Negative	0.8333	0.7282	0.7772	4,827
Neutral	0.8046	0.8764	0.8390	7,170
Positive	0.8728	0.8699	0.8714	9,419
Accuracy			0.8402	21,416
Macro avg.	0.8369	0.8249	0.8292	21,416
Weighted avg.	0.8411	0.8402	0.8393	21,416

Table 8.4: GRU Classification Report

Category	Precision	Recall	F1-score	Support
Negative	0.8227	0.7354	0.7766	4,827
Neutral	0.8272	0.8612	0.8439	7,170
Positive	0.8565	0.8762	0.8662	9,419
Accuracy			0.8395	21,416
Macro avg.	0.8355	0.8243	0.8289	21,416
Weighted avg.	0.8391	0.8395	0.8385	21,416

Table 8.5: DistilBERT Classification Report

Category	Precision	Recall	F1-score	Support
Negative	0.8414	0.8179	0.8295	4,827
Neutral	0.8435	0.8893	0.8658	7,170
Positive	0.9114	0.8868	0.8989	9,419
Accuracy			0.8721	21,416
Macro avg.	0.8654	0.8647	0.8647	21,416
Weighted avg.	0.8729	0.8721	0.8722	21,416

Every model finds the negative class the hardest, with recall of 67.4% (Simple RNN), 72.8% (LSTM), 73.5% (GRU) and 81.8% (DistilBERT). This is the smallest class and its comments are often confused with neutral ones. The three recurrent models have a recall of only 67–74% on negative comments but 84–88% on neutral and positive comments, i.e. they are biased toward the larger classes. DistilBERT reduces this gap: its negative-class F1-score is 5.2 points higher than the LSTM's, compared with gains of about 2.7 points for the neutral and positive classes.

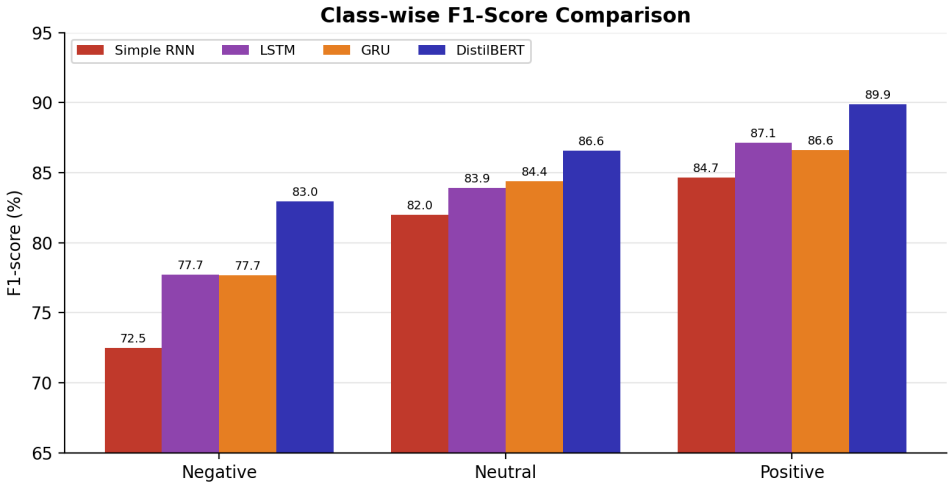


Figure 8.3: Class-wise F1-Score Comparison

8.3 Validation Performance

Validation accuracy and loss show the training behaviour of the three Keras models; DistilBERT is excluded because it is trained in a separate PyTorch loop and monitored by validation F1-score.

Table 8.6: Validation Accuracy and Loss per Epoch (Keras Models)

Model	Validation Accuracy (%)				Validation Loss			
	E1	E2	E3	E4	E1	E2	E3	E4
Simple RNN	81.51	81.69	80.82	79.75	0.541	0.522	0.575	0.705
LSTM	84.03	84.13	83.91	83.03	0.445	0.439	0.469	0.545
GRU	84.11	84.01	–	–	0.448	0.446	–	–

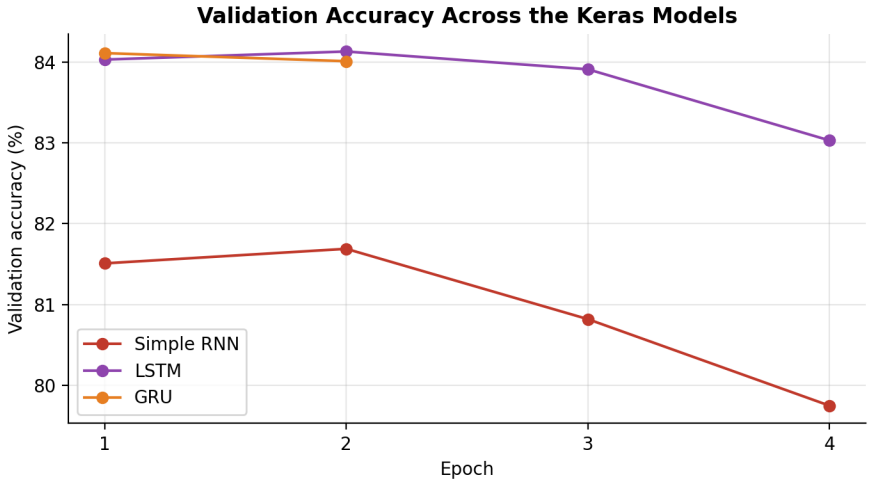


Figure 8.4: Validation Accuracy Across the Keras Models

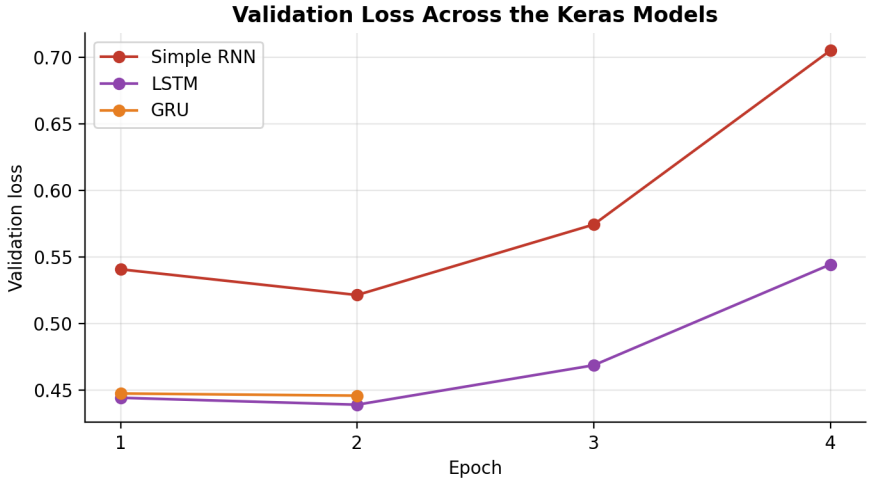


Figure 8.5: Validation Loss Across the Keras Models

The LSTM and GRU keep validation accuracy about 2.3 to 3.3 points above the Simple RNN at every epoch. For the Simple RNN and LSTM, validation accuracy peaks and validation loss is lowest at epoch 2, after which the loss rises sharply (to 0.705 and 0.545 at epoch 4) even though training accuracy keeps increasing. The

models begin to memorize the training comments rather than learn generalizable patterns, so restoring the best-epoch weights is essential.

8.4 Confusion Matrix Analysis

Figure 8.6 shows the confusion matrices; rows are actual classes and columns are predicted classes.

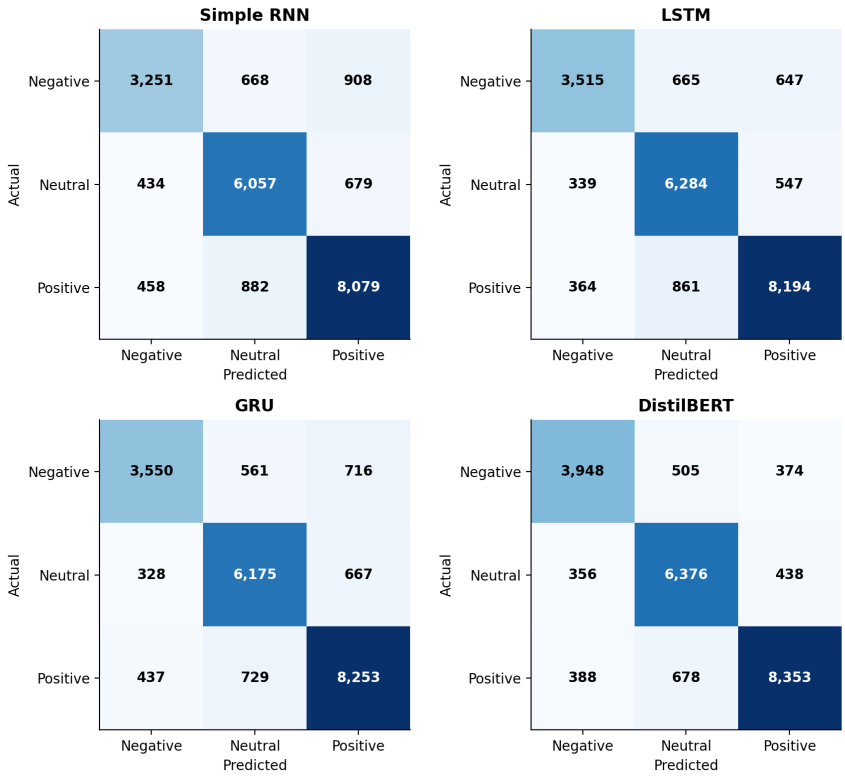


Figure 8.6: Confusion Matrix Comparison of the Four Models

Table 8.7: Error Decomposition from the Confusion Matrices

Model	Correct	Errors	Polarity flips	Neutral-related	Neutral share
Simple RNN	17,387	4,029	1,366	2,663	66.1%
LSTM	17,993	3,423	1,011	2,412	70.5%
GRU	17,978	3,438	1,153	2,285	66.5%
DistilBERT	18,677	2,739	762	1,977	72.2%

Here "polarity flips" are comments whose predicted label is the opposite polar class (negative predicted as positive or vice versa), and "neutral-related" errors are those in which either the actual or the predicted class is neutral. Several patterns

appear. First, DistilBERT makes 2,739 errors against 3,423 for the LSTM, a reduction of about 20%. Second, a large majority of all errors for every model (66–72%) involve the neutral class, so most mistakes are "soft" confusions between a polar and a neutral comment. Third, the share of severe errors that flip negative to positive or vice versa is smallest for DistilBERT (762 comments, 3.6% of the test set) and largest for the Simple RNN (1,366 comments, 6.4%). Fourth, DistilBERT's most frequent single error is positive comments predicted as neutral (678 comments), and negative comments predicted as neutral (505) is second, which shows that the boundary between neutral and polar sentiment remains the main difficulty.

8.5 Training Time, Inference Time, and Model Parameters

Table 8.8: Computational Performance of the Four Models

Model	Epochs	Training (s)	Per epoch (s)	Inference (s)	Parameters
Simple RNN	4	86.27	21.57	1.70	2,572,547
LSTM	4	69.22	17.31	0.98	2,609,603
GRU	2	27.90	13.95	0.91	2,597,443
DistilBERT	2	4,394.10	2,197.05	84.83	66,955,779

8.5.1 Training Time Comparison

DistilBERT needs 4,394 seconds (about 73 minutes) to fine-tune, against 27.9–86.3 seconds for the recurrent models. Total times are not directly comparable because the models ran for different numbers of epochs. Per epoch, the GRU is fastest (13.95 s), followed by the LSTM (17.31 s) and the Simple RNN (21.57 s); the Simple RNN is slowest of the three probably because Keras has an optimized GPU (cuDNN) kernel for LSTM and GRU layers but not for the Simple RNN. Per epoch, DistilBERT is about 157 times slower than the GRU and 127 times slower than the LSTM.

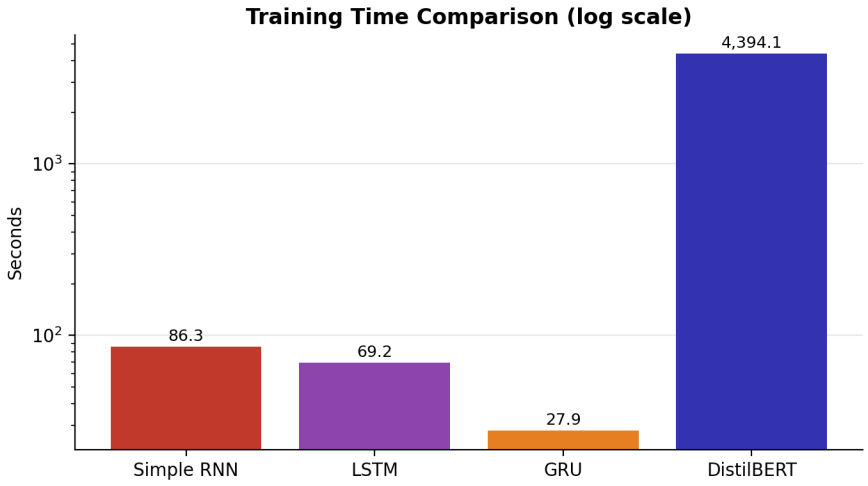


Figure 8.7: Training Time Comparison of the Four Models

8.5.2 Inference Time Comparison

Predicting the 21,416 test comments takes 0.91 s for the GRU and 0.98 s for the LSTM, but 84.83 s for DistilBERT, about 93 times longer than the GRU. This is a throughput of roughly 23,000 comments per second for the GRU against about 250 for DistilBERT.

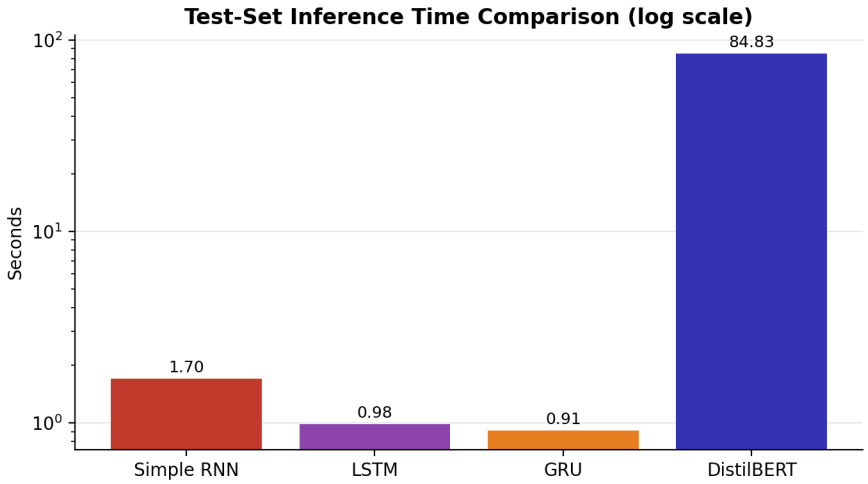


Figure 8.8: Test-Set Inference Time Comparison of the Four Models

8.5.3 Model Parameter Comparison

The three recurrent models have almost the same size, about 2.6 million parameters, of which 2.56 million belong to the embedding table. DistilBERT has 66.96 million parameters, about 26 times as many.

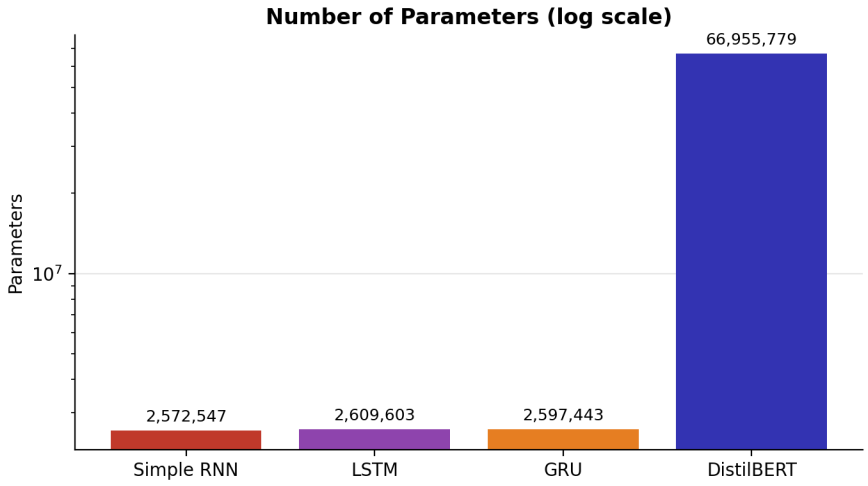


Figure 8.9: Number of Parameters in Each Model

8.6 DistilBERT Training Progress

DistilBERT's training loss falls from 0.5068 to 0.3432 between the two epochs, and its validation weighted F1-score improves from 0.8616 to 0.8722, so the second epoch gives the best checkpoint. Because validation performance was still rising when training ended, additional epochs might improve it slightly.

Table 8.9: DistilBERT Training Loss and Validation F1-Score

Epoch	Training Loss	Validation Weighted F1
1	0.5068	0.8616
2	0.3432	0.8722

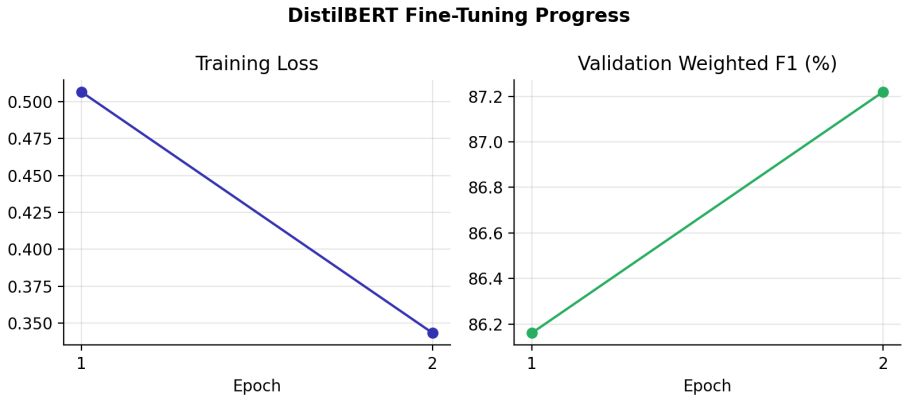


Figure 8.10: DistilBERT Fine-Tuning Progress

8.7 Comparative Analysis

- **DistilBERT** has the highest accuracy (87.21%), weighted F1-score (87.22%) and macro F1-score (86.47%), the best recall on the difficult negative class (81.8%) and the fewest severe errors, but it is the most expensive model in time and size.
- **LSTM and GRU** perform equally well (84.02% and 83.95%). The GRU is faster per epoch and has fewer parameters, so it offers the best balance of accuracy and cost among the recurrent models.
- **Simple RNN** is the least accurate (81.19%) and the slowest recurrent model per epoch, so the gating in LSTM and GRU brings both better accuracy and faster training here.
- The results show a clear **trade-off between predictive performance and computational cost**.

Chapter 9

Sample Predictions

9.1 Sample Comment Predictions

The trained models were used to predict the sentiment of three new comments written for illustration: one clearly positive, one ambiguous and one clearly negative. Each entry shows the predicted class and the model's confidence (the softmax probability of that class).

Table 9.1: Sample Sentiment Prediction Results

Comment	Simple RNN	LSTM	GRU	DistilBERT
The product was excellent and I really enjoyed using it.	Positive (96.59%)	Positive (98.23%)	Positive (92.23%)	Positive (99.45%)
It was okay, nothing particularly special.	Positive (94.50%)	Positive (49.77%)	Positive (68.44%)	**Neutral** (39.92%)
The experience was terrible and very disappointing.	Negative (98.24%)	Negative (97.52%)	Negative (95.96%)	Negative (98.40%)

All four models agree on the clearly positive and clearly negative comments and assign them high confidence. The ambiguous comment separates the models: the three recurrent models label it Positive, the Simple RNN with misplaced confidence of 94.50%, whereas DistilBERT alone recognizes it as Neutral.

9.2 Interpretation of the Neutral Example

The comment "It was okay, nothing particularly special" contains a mildly positive word ("okay") and a mildly negative phrase ("nothing particularly special"). A model that relies mainly on individual word cues leans toward the positive word, which is what the recurrent models do. DistilBERT, whose self-attention and pre-training let it weigh the whole phrase in context, predicts Neutral, but with a confidence of only

39.92%, which shows that neutral sentiment remains hard even for the best model. The low confidence of the LSTM (49.77%) and of DistilBERT is useful in practice: it signals an uncertain prediction that could be flagged for human review.

Chapter 10

Discussion

10.1 Comparison of Model Architectures

The results show a consistent ordering: the Simple RNN is weakest, the LSTM and GRU are intermediate and equal, and the pre-trained DistilBERT is strongest. The Simple RNN struggles because gradients vanish over long sequences, and its validation loss deteriorates fastest. The gates of the LSTM and GRU keep useful information over longer spans, which gives about 2.8 points of accuracy. That the GRU matches the LSTM with fewer parameters suggests that, for short comments, the extra gate and cell state of the LSTM add little.

DistilBERT's advantage of about 3 points comes from three factors: self-attention, which relates every token to every other token regardless of distance; contextual representations, in which a word's vector depends on its neighbours; and, above all, large-scale pre-training, which gives it knowledge of language that the recurrent models, whose embeddings are learned only from 171,325 training comments, do not have. Part of the gap is therefore due to pre-training rather than to the architecture alone, which is an important caveat when interpreting the comparison.

10.2 Interpretation of Classification Errors

Most errors of all models involve the neutral class (66–72%). Neutral comments have no sharp lexical signature and lie between the two polar classes, so annotators, and models, can disagree about them. The negative class is the second source of difficulty: it has the fewest training examples (22.5%), and the recurrent models recover only 67–74% of negative comments, which places their macro F1-scores about 1.0 to 1.3 points below their weighted F1-scores (0.75 points for DistilBERT). DistilBERT's pre-trained knowledge helps most here, raising negative recall to 81.8%. Its remaining errors are mostly positive or negative comments labelled neutral, which are less harmful in practice than a complete reversal of polarity; such reversals account for only 27.8% of DistilBERT's errors.

10.3 Predictive Performance and Computational Requirements

DistilBERT improves accuracy by about 3.2 points over the best recurrent model, but at roughly 26 times the parameters, about 127 to 157 times the training time per epoch and about 90 times the inference time. Whether this cost is justified depends on the application. Where each percentage point of accuracy is valuable and the volume is moderate, as in offline analysis of customer feedback, DistilBERT is the better choice. For real-time processing of very large streams, or on limited hardware, the GRU or LSTM delivers 84% accuracy at a small fraction of the cost, and the GRU in particular offers the best accuracy per unit of computation among the recurrent models. These considerations also suggest that a hybrid design could be effective, in which a fast recurrent model handles confident cases and uncertain ones are passed to DistilBERT.

10.4 Interpretation of the Final Model Selection

DistilBERT is the best-performing model on every metric and is the recommended model when accuracy is the priority. In this study the validation subset, not the test subset, was used for early stopping and checkpoint selection, so the test results are an unbiased estimate of performance for the selected checkpoints. The conclusions nevertheless rest on a single training run per model, so small differences (such as that between the LSTM and GRU) should not be over-interpreted, whereas the large gaps between DistilBERT and the recurrent models and between the Simple RNN and the gated models are substantial relative to the size of the test set.

Chapter 11

Limitations

The study has the following limitations:

1. **Single dataset and language.** The experiments use one English-language dataset of user comments, many of which concern Apple Pay and contactless payment. The results may not carry over to other domains, languages or writing styles.

2. **Single experimental run.** Each model was trained once with one random seed, so run-to-run variation is not measured and no statistical significance test is applied. Differences below about half a percentage point, such as that between the LSTM and GRU, should be treated as ties.

3. **Limited hyperparameter tuning.** Embedding size, recurrent units, dropout, learning rate and sequence length were fixed in advance and not systematically optimized. DistilBERT was trained for only two epochs, and its validation F1-score was still improving.

4. **Uneven training of the GRU.** The GRU run ended after two epochs, fewer than the RNN and LSTM, so its total training time is not directly comparable; per-epoch time is reported to compensate.

5. **Unequal comparison conditions.** DistilBERT is pre-trained on large corpora, while the recurrent models learn their embeddings from scratch; the models also run in different frameworks (PyTorch and TensorFlow) with different batch sizes. The performance gap therefore reflects pre-training as well as architecture, and timings indicate order of magnitude only.

6. **Class imbalance.** The negative class makes up only about 22.5% of the data, and no class weighting or resampling was used. All models are weakest on this class.

7. **Ambiguity of the neutral class.** Neutral labels are inherently subjective, and the quality of the labels was not independently verified.

8. **Truncation and light preprocessing.** Comments are truncated to 128 tokens, which may drop information from long comments, and the supplied text was already cleaned, so the models never see raw, noisy text.

9. **Timing environment.** Training and inference times depend on the Colab GPU allocation and system load.

10. **Small illustrative prediction set.** The sample predictions use only three comments and illustrate behaviour; they do not measure it.

Chapter 12

Future Enhancements

Possible directions for extending this work are:

1. **Repeated runs and significance testing.** Train each model with several random seeds and apply McNemar's test or bootstrap confidence intervals so that small differences can be judged reliably.

2. **Hyperparameter optimization.** Search over embedding size, hidden units, dropout, learning rate, sequence length and the number of DistilBERT epochs.

3. **A from-scratch Transformer encoder.** Add a custom Transformer with multi-head self-attention to separate the effect of the attention architecture from the effect of pre-training.

4. **Pre-trained word embeddings.** Initialize the recurrent models with GloVe or fastText vectors for a fairer comparison with DistilBERT.

5. **Handling class imbalance.** Use class weights, focal loss or resampling to improve recall on the negative class.

6. **Other pre-trained models.** Fine-tune BERT, RoBERTa or domain-specific models such as BERTweet and compare them with DistilBERT.

7. **Faster fine-tuning.** Apply mixed-precision training, dynamic padding and larger batches to reduce DistilBERT's training time.

8. **Model compression.** Use quantization, pruning or further distillation to reduce inference cost for deployment.

9. **Confidence-based routing.** Combine a fast recurrent model with DistilBERT, sending low-confidence comments to the larger model.

10. **Systematic error analysis.** Inspect misclassified comments, especially neutral-versus-polar cases, to understand failure modes and label noise.

11. **Deployment.** Build a web interface for real-time sentiment prediction, and extend the system to multilingual and finer-grained (e.g. five-class) sentiment.

Chapter 13

Conclusion

This case study compared four deep learning models for three-class sentiment classification of user comments: a Simple RNN, an LSTM, a GRU and a fine-tuned DistilBERT. A dataset of 241,145 raw comments was cleaned to 214,157 unique comments and divided into stratified training, validation and test subsets of 171,325, 21,416 and 21,416 comments.

DistilBERT achieved the highest test accuracy of 87.21%, with a weighted F1-score of 87.22% and a macro F1-score of 86.47%. The LSTM and GRU performed almost identically (84.02% and 83.95%), and the Simple RNN reached 81.19%. All models were weakest on the minority negative class, and most of their errors involved the neutral class; DistilBERT recovered the most negative comments (81.8% recall) and made the fewest polarity-reversing errors.

The cost of DistilBERT's accuracy was high: about 26 times as many parameters, roughly 127 to 157 times the training time per epoch and about 90 times the inference time of the recurrent models. The GRU achieved nearly the accuracy of the LSTM with the lowest per-epoch training time and inference time, making it the most efficient recurrent model. Sample predictions showed that DistilBERT alone recognized an ambiguous comment as neutral.

The study highlights the trade-off between classification performance and computational efficiency. Pre-trained Transformers are the better choice when accuracy matters most and resources allow, while compact gated recurrent models are practical when speed and low cost are required. Repeated runs, hyperparameter tuning, imbalance handling and a from-scratch Transformer are natural next steps.

Appendix A

Source Code

The complete Python code used for the experiments is listed below. It was executed as a Google Colab notebook; the cells appear in execution order.

```
# ---------- Cell 1 ----------
# Run once in Google Colab. (TensorFlow is already installed in Colab; we do NOT upgrade it,
# which avoids the dependency-conflict errors.)

# ---------- Cell 2 ----------
import os, re, time, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_recall_fscore_support,
    classification_report, confusion_matrix
)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))

# ---------- Cell 3 ----------
DATA_PATH = "/content/sentiment_data.csv"

# If the file is not found, ask for an upload (Colab only)
if not os.path.exists(DATA_PATH):
    try:
        from google.colab import files
        print("Please upload sentiment_data.csv")
        uploaded = files.upload()
        DATA_PATH = list(uploaded.keys())[0]
    except ImportError:
        DATA_PATH = "sentiment_data.csv"   # local run

df = pd.read_csv(DATA_PATH)

print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
```

```
print("\nMissing values:")
print(df.isna().sum())
display(df.head())

# ---------- Cell 4 ----------
df = df[["Comment", "Sentiment"]].copy()

# Remove rows with missing values
df = df.dropna(subset=["Comment", "Sentiment"])

# Remove duplicate comments (done BEFORE splitting to prevent train/test leakage)
df = df.drop_duplicates(subset=["Comment"]).reset_index(drop=True)

df["Comment"] = df["Comment"].astype(str).str.strip()
df["Sentiment"] = df["Sentiment"].astype(int)

# Keep only the three expected labels
df = df[df["Sentiment"].isin([0, 1, 2])].reset_index(drop=True)

label_names = {0: "Negative", 1: "Neutral", 2: "Positive"}

print("Cleaned shape:", df.shape)
print("\nClass distribution:")
print(df["Sentiment"].value_counts().sort_index())

display(df["Sentiment"].value_counts().sort_index().rename(index=label_names).to_frame("count"))

# ---------- Cell 5 ----------
counts = df["Sentiment"].value_counts().sort_index()

plt.figure(figsize=(7, 4))
plt.bar([label_names[i] for i in counts.index], counts.values)
plt.title("Sentiment Class Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Comments")
plt.tight_layout()
plt.show()

# ---------- Cell 6 ----------
train_df, temp_df = train_test_split(
    df, test_size=0.20, stratify=df["Sentiment"], random_state=SEED
)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, stratify=temp_df["Sentiment"], random_state=SEED
)

print("Train:", len(train_df))
print("Validation:", len(val_df))
print("Test:", len(test_df))

for name, d in [("Train", train_df), ("Validation", val_df), ("Test", test_df)]:
    print(f"\n{name} distribution:")
    print(d["Sentiment"].value_counts(normalize=True).sort_index())

# ---------- Cell 7 ----------
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"<br\s*/?>", " ", text)
    text = re.sub(r"http\S+|www\.\S+", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

train_texts = train_df["Comment"].map(clean_text).tolist()
```

```
val_texts   = val_df["Comment"].map(clean_text).tolist()
test_texts  = test_df["Comment"].map(clean_text).tolist()

y_train = train_df["Sentiment"].to_numpy()
y_val   = val_df["Sentiment"].to_numpy()
y_test  = test_df["Sentiment"].to_numpy()

print(train_texts[:2])

# ---------- Cell 8 ----------
VOCAB_SIZE = 20000
MAX_LEN = 128
EMBED_DIM = 128

tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>")

# Fit only on training text to avoid vocabulary leakage
tokenizer.fit_on_texts(train_texts)

def to_padded(texts):
    seqs = tokenizer.texts_to_sequences(texts)
    return tf.keras.preprocessing.sequence.pad_sequences(
        seqs, maxlen=MAX_LEN, padding="post", truncating="post"
    )

X_train = to_padded(train_texts)
X_val   = to_padded(val_texts)
X_test  = to_padded(test_texts)

actual_vocab_size = min(VOCAB_SIZE, len(tokenizer.word_index) + 1)

print("Vocabulary used:", actual_vocab_size)
print("Train shape:", X_train.shape)
print("Validation shape:", X_val.shape)
print("Test shape:", X_test.shape)

# ---------- Cell 9 ----------
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=2, restore_best_weights=True
)
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss", factor=0.5, patience=1, min_lr=1e-6
)

BATCH_SIZE = 128
EPOCHS = 5
NUM_CLASSES = 3

results = []
trained_models = {}
histories = {}

def evaluate_keras_model(model, model_name, history, X_test, y_test, training_time):
    start = time.perf_counter()
    probs = model.predict(X_test, batch_size=BATCH_SIZE, verbose=0)
    inference_time = time.perf_counter() - start
    preds = np.argmax(probs, axis=1)

    acc = accuracy_score(y_test, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test, preds, average="weighted", zero_division=0
    )
    macro_f1 = precision_recall_fscore_support(
```

```
        y_test, preds, average="macro", zero_division=0
    )[2]

    epochs_run = len(history.history["loss"])

    results.append({
        "Model": model_name,
        "Accuracy": acc,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1,
        "Macro F1": macro_f1,
        "Epochs Run": epochs_run,
        "Training Time (s)": training_time,
        "Time / Epoch (s)": training_time / epochs_run,
        "Inference Time (s)": inference_time,
        "Parameters": model.count_params()
    })
    trained_models[model_name] = model
    histories[model_name] = history.history

    print(f"\n{model_name}  (epochs run: {epochs_run})")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall   : {recall:.4f}")
    print(f"F1-score : {f1:.4f}   |   Macro F1: {macro_f1:.4f}")
    print("\nClassification Report:")
    print(classification_report(
        y_test, preds,
        target_names=[label_names[i] for i in range(NUM_CLASSES)],
        digits=4, zero_division=0
    ))
    return preds, probs

def build_recurrent(layer_cls, units=64):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(MAX_LEN,)),
        tf.keras.layers.Embedding(input_dim=actual_vocab_size, output_dim=EMBED_DIM, mask_zero=
     True),
        layer_cls(units),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.Dense(NUM_CLASSES, activation="softmax")
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

# ---------- Cell 10 ----------
tf.keras.backend.clear_session()
tf.random.set_seed(SEED)

rnn = build_recurrent(tf.keras.layers.SimpleRNN)
rnn.summary()

start = time.perf_counter()
history_rnn = rnn.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    callbacks=[early_stop, reduce_lr], verbose=1
```

```
)
rnn_training_time = time.perf_counter() - start

rnn_preds, rnn_probs = evaluate_keras_model(
    rnn, "Simple RNN", history_rnn, X_test, y_test, rnn_training_time
)

# ---------- Cell 11 ----------
tf.random.set_seed(SEED)

lstm = build_recurrent(tf.keras.layers.LSTM)
lstm.summary()

start = time.perf_counter()
history_lstm = lstm.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    callbacks=[early_stop, reduce_lr], verbose=1
)
lstm_training_time = time.perf_counter() - start

lstm_preds, lstm_probs = evaluate_keras_model(
    lstm, "LSTM", history_lstm, X_test, y_test, lstm_training_time
)

# ---------- Cell 12 ----------
tf.random.set_seed(SEED)

gru = build_recurrent(tf.keras.layers.GRU)
gru.summary()

start = time.perf_counter()
history_gru = gru.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    callbacks=[early_stop, reduce_lr], verbose=1
)
gru_training_time = time.perf_counter() - start

gru_preds, gru_probs = evaluate_keras_model(
    gru, "GRU", history_gru, X_test, y_test, gru_training_time
)

# ---------- Cell 13 ----------
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification, get_linear_schedule_with_warmup
)

torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch device:", device)

BERT_MODEL_NAME = "distilbert-base-uncased"
BERT_MAX_LEN = 128
BERT_BATCH_SIZE = 16
BERT_EPOCHS = 2
```

```
BERT_LR = 2e-5

bert_tokenizer = AutoTokenizer.from_pretrained(BERT_MODEL_NAME)

# ---------- Cell 14 ----------
class SentimentDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        enc = tokenizer(
            list(texts), truncation=True, padding="max_length",
            max_length=max_len, return_tensors="pt"
        )
        self.input_ids = enc["input_ids"]
        self.attention_mask = enc["attention_mask"]
        self.labels = torch.tensor(list(labels), dtype=torch.long)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return {
            "input_ids": self.input_ids[idx],
            "attention_mask": self.attention_mask[idx],
            "labels": self.labels[idx]
        }

bert_train_ds = SentimentDataset(train_texts, y_train, bert_tokenizer, BERT_MAX_LEN)
bert_val_ds   = SentimentDataset(val_texts,   y_val,   bert_tokenizer, BERT_MAX_LEN)
bert_test_ds  = SentimentDataset(test_texts,  y_test,  bert_tokenizer, BERT_MAX_LEN)

pin = torch.cuda.is_available()
bert_train_loader = DataLoader(bert_train_ds, batch_size=BERT_BATCH_SIZE, shuffle=True,
    pin_memory=pin)
bert_val_loader   = DataLoader(bert_val_ds,   batch_size=BERT_BATCH_SIZE, shuffle=False,
    pin_memory=pin)
bert_test_loader  = DataLoader(bert_test_ds,  batch_size=BERT_BATCH_SIZE, shuffle=False,
    pin_memory=pin)

print(len(bert_train_ds), len(bert_val_ds), len(bert_test_ds))

# ---------- Cell 15 ----------
bert_model = AutoModelForSequenceClassification.from_pretrained(
    BERT_MODEL_NAME, num_labels=NUM_CLASSES
)
bert_model.to(device)

optimizer = torch.optim.AdamW(bert_model.parameters(), lr=BERT_LR)

total_training_steps = len(bert_train_loader) * BERT_EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_training_steps),
    num_training_steps=total_training_steps
)

print("Trainable parameters:", sum(p.numel() for p in bert_model.parameters() if p.requires_grad))

# ---------- Cell 16 ----------
def evaluate_bert(model, loader):
    model.eval()
    all_preds, all_labels = [], []
    with torch.no_grad():
        for batch in loader:
            input_ids = batch["input_ids"].to(device)
```

```
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(input_ids=input_ids, attention_mask=attention_mask)
            preds = torch.argmax(outputs.logits, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    return np.array(all_labels), np.array(all_preds)

best_val_f1 = -1.0
best_state = None
bert_history = {"train_loss": [], "val_f1": []}

start = time.perf_counter()

for epoch in range(BERT_EPOCHS):
    bert_model.train()
    total_loss = 0.0

    for batch in bert_train_loader:
        optimizer.zero_grad(set_to_none=True)

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = bert_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()

        loss.backward()
        torch.nn.utils.clip_grad_norm_(bert_model.parameters(), max_norm=1.0)
        optimizer.step()
        scheduler.step()

    val_labels, val_preds = evaluate_bert(bert_model, bert_val_loader)
    val_f1 = precision_recall_fscore_support(
        val_labels, val_preds, average="weighted", zero_division=0
    )[2]
    avg_loss = total_loss / len(bert_train_loader)

    bert_history["train_loss"].append(avg_loss)
    bert_history["val_f1"].append(val_f1)
    print(f"Epoch {epoch+1}/{BERT_EPOCHS} | Train Loss: {avg_loss:.4f} | Val F1: {val_f1:.4f}")

    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        best_state = {k: v.detach().cpu().clone() for k, v in bert_model.state_dict().items()}

bert_training_time = time.perf_counter() - start

# Restore best validation checkpoint
bert_model.load_state_dict(best_state)
bert_model.to(device)
print(f"\nDistilBERT training time: {bert_training_time:.1f} s")

# ---------- Cell 17 ----------
# Evaluate DistilBERT on the untouched test set
start = time.perf_counter()
test_labels_bert, test_preds_bert = evaluate_bert(bert_model, bert_test_loader)
bert_inference_time = time.perf_counter() - start

```

```
bert_accuracy = accuracy_score(test_labels_bert, test_preds_bert)
bert_precision, bert_recall, bert_f1, _ = precision_recall_fscore_support(
    test_labels_bert, test_preds_bert, average="weighted", zero_division=0
)
bert_macro_f1 = precision_recall_fscore_support(
    test_labels_bert, test_preds_bert, average="macro", zero_division=0
)[2]
bert_parameters = sum(p.numel() for p in bert_model.parameters())

results.append({
    "Model": "DistilBERT",
    "Accuracy": bert_accuracy,
    "Precision": bert_precision,
    "Recall": bert_recall,
    "F1-Score": bert_f1,
    "Macro F1": bert_macro_f1,
    "Epochs Run": BERT_EPOCHS,
    "Training Time (s)": bert_training_time,
    "Time / Epoch (s)": bert_training_time / BERT_EPOCHS,
    "Inference Time (s)": bert_inference_time,
    "Parameters": bert_parameters
})
trained_models["DistilBERT"] = bert_model

print("DistilBERT Results")
print(f"Accuracy : {bert_accuracy:.4f}")
print(f"Precision: {bert_precision:.4f}")
print(f"Recall   : {bert_recall:.4f}")
print(f"F1-score : {bert_f1:.4f}   |   Macro F1: {bert_macro_f1:.4f}")
print("\nClassification Report:")
print(classification_report(
    test_labels_bert, test_preds_bert,
    target_names=[label_names[i] for i in range(NUM_CLASSES)],
    digits=4, zero_division=0
))

# ---------- Cell 18 ----------
results_df = pd.DataFrame(results).sort_values("Accuracy", ascending=False).reset_index(drop=True)
display(results_df)

results_df.to_csv("model_comparison_results.csv", index=False)

# ---------- Cell 19 ----------
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, metric in zip(axes, ["Accuracy", "F1-Score"]):
    bars = ax.bar(results_df["Model"], results_df[metric] * 100)
    ax.set_ylim(70, 95)   # zoomed axis so the differences are visible
    ax.set_ylabel(f"{metric} (%)")
    ax.set_xlabel("Model")
    ax.set_title(f"{metric} Comparison of Four Models")
    ax.tick_params(axis="x", rotation=20)
    for b, v in zip(bars, results_df[metric] * 100):
        ax.text(b.get_x() + b.get_width()/2, v + 0.2, f"{v:.2f}", ha="center")

plt.tight_layout()
plt.show()

# ---------- Cell 20 ----------
# Computational comparison
display(results_df[["Model", "Epochs Run", "Training Time (s)", "Time / Epoch (s)",
                    "Inference Time (s)", "Parameters"]])

```

```
# ---------- Cell 21 ----------
# Clean table for the presentation
presentation_table = results_df.copy()

for col in ["Accuracy", "Precision", "Recall", "F1-Score", "Macro F1"]:
    presentation_table[col] = (presentation_table[col] * 100).round(2).astype(str) + "%"

for col in ["Training Time (s)", "Time / Epoch (s)", "Inference Time (s)"]:
    presentation_table[col] = presentation_table[col].round(2)

display(presentation_table)

# ---------- Cell 22 ----------
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

for name, h in histories.items():
    axes[0].plot(range(1, len(h["val_loss"]) + 1), h["val_loss"], marker="o", label=name)
    axes[1].plot(range(1, len(h["val_accuracy"]) + 1), h["val_accuracy"], marker="o", label=name)

axes[0].set_title("Validation Loss"); axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss");
     axes[0].legend()
axes[1].set_title("Validation Accuracy"); axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("
     Accuracy"); axes[1].legend()

plt.tight_layout()
plt.show()

# ---------- Cell 23 ----------
prediction_dict = {
    "Simple RNN": rnn_preds,
    "LSTM": lstm_preds,
    "GRU": gru_preds,
    "DistilBERT": test_preds_bert
}

fig, axes = plt.subplots(2, 2, figsize=(11, 9))
axes = axes.flatten()
tick_labels = ["Negative", "Neutral", "Positive"]

for ax, (name, preds) in zip(axes, prediction_dict.items()):
    cm = confusion_matrix(y_test, preds, labels=[0, 1, 2])
    ax.imshow(cm, cmap="Blues")
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_xticks([0, 1, 2]); ax.set_yticks([0, 1, 2])
    ax.set_xticklabels(tick_labels); ax.set_yticklabels(tick_labels)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                    color="white" if cm[i, j] > cm.max() / 2 else "black")

plt.tight_layout()
plt.show()

# ---------- Cell 24 ----------
def predict_custom(model, text):
    cleaned = clean_text(text)
    seq = tokenizer.texts_to_sequences([cleaned])
    seq = tf.keras.preprocessing.sequence.pad_sequences(
        seq, maxlen=MAX_LEN, padding="post", truncating="post"
    )
    probs = model.predict(seq, verbose=0)[0]
```

```
    pred = int(np.argmax(probs))
    return label_names[pred], float(probs[pred])

def predict_distilbert(text):
    bert_model.eval()
    encoding = bert_tokenizer(
        text, truncation=True, padding="max_length",
        max_length=BERT_MAX_LEN, return_tensors="pt"
    )
    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)

    with torch.no_grad():
        outputs = bert_model(input_ids=input_ids, attention_mask=attention_mask)

    probs = torch.softmax(outputs.logits, dim=1)[0]
    pred = int(torch.argmax(probs).item())
    return label_names[pred], float(probs[pred].item())

# ---------- Cell 25 ----------
sample_comments = [
    "The product was excellent and I really enjoyed using it.",
    "It was okay, nothing particularly special.",
    "The experience was terrible and very disappointing."
]

for comment in sample_comments:
    print("\nComment:", comment)

    for name, model in [("Simple RNN", rnn), ("LSTM", lstm), ("GRU", gru)]:
        sentiment, confidence = predict_custom(model, comment)
        print(f"{name:20s}: {sentiment:8s} ({confidence*100:.2f}%)")

    sentiment, confidence = predict_distilbert(comment)
    print(f"{'DistilBERT':20s}: {sentiment:8s} ({confidence*100:.2f}%)")

# ---------- Cell 26 ----------
test_eval_df = test_df.reset_index(drop=True).copy()
test_eval_df["Predicted"] = test_preds_bert
wrong = test_eval_df[test_eval_df["Sentiment"] != test_eval_df["Predicted"]].copy()
wrong["Actual"] = wrong["Sentiment"].map(label_names)
wrong["Predicted"] = wrong["Predicted"].map(label_names)

print(f"Misclassified: {len(wrong)} of {len(test_eval_df)} ({len(wrong)/len(test_eval_df)*100:.2f
    }%)")
pd.set_option("display.max_colwidth", 140)
display(wrong[["Comment", "Actual", "Predicted"]].sample(10, random_state=SEED))

# ---------- Cell 27 ----------
os.makedirs("saved_models", exist_ok=True)

rnn.save("saved_models/simple_rnn.keras")
lstm.save("saved_models/lstm.keras")
gru.save("saved_models/gru.keras")

bert_model.save_pretrained("saved_models/distilbert_sentiment")
bert_tokenizer.save_pretrained("saved_models/distilbert_sentiment")

print("All four models saved.")
```

Bibliography

[1] Vaswani, A., et al. (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems, 30.

[2] Devlin, J., et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* Proceedings of NAACL-HLT, pp. 4171–4186.

[3] Sanh, V., et al. (2019). *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter.* arXiv:1910.01108.

[4] Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

[5] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. Proceedings of EMNLP.

[6] Kingma, D. P., and Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.

[7] Loshchilov, I., and Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.

[8] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP: System Demonstrations.